

Weight Initialization Techniques:

Weight Initialization is the process of setting the starting values for NN weights before training. These weights help the network learn by adjusting them thru each iteration.

Poor Weight Initialization leads to

- * Exploding gradient Problem
- * Vanishing gradient Problem
- * Slower Convergence.

So proper initialization reduce these problems, making the network converge faster & perform better.

Types of Weight Initialization Techniques

(i) Zero Initialization:

This approach sets all weights to zero.

It's rarely used coz it causes neuron to learn the same features, making them "indistinguishable".

In other words, the model won't learn anything meaningful.

(ii) Random Initialization:

This approach randomly assigns weights to avoid zero initialization problem.

However if we choose values that are too small (or) too large, we face other issues.

Small Random Weights

Values that are too small causes "vanishing gradient problem", especially in deep networks. This results in extremely slow training.

Large Random Weights;

Values that are large cause "Exploding gradient problem", where gradients become excessively large. This causes instability in training.

Initialization can be two kinds.

$w_i \sim N(0, 1) \Rightarrow$ Initialized values from normal distribution. (Random Normal)

$w_i \sim U(0, 1) \Rightarrow$ from uniform distribution (Random uniform)

(iii) Xavier Initialization (Glorot Initialization)

This approach aims to keep variance of activations constant across layers. This method assigns weights from a normal distribution with mean 0 and a specific variance based on num of neurons in each layer. This initialization is particularly useful when tanh or sigmoid activation function.

Example: Weights are distributed so neither vanishing / exploding gradient occurs, creates a balanced learning environment for each layer.

1.) Normalized Xavier Initialization:

$$w_i \sim N(0, \sigma) \quad (\text{suitable for Sigmoid / tanh})$$

$$\sigma = \sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$$

2.) Uniform Xavier Initialization:

$$w_i \sim U \left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}} \right]$$

(iv) He Initialization:

Similar to Xavier but better suited for layers with ReLU activation functions.

Scales the weights by square root of num of neurons multiplied by 2. This helps prevent exploding gradient problem and ensuring layers using ReLU "do not die out".

From uniform Distribution:

$$w_i \sim U - \left[\sqrt{\frac{6}{\text{fan-in}}}, \sqrt{\frac{6}{\text{fan-out}}} \right]$$

From Normal Distribution:

$$w_i \sim N[0, \sigma]$$
$$\sigma = \sqrt{\frac{2}{\text{fan-in}}}$$

Summary:

Sigmoid/tanh $\rightarrow$ Xavier/Glorot

ReLU $\rightarrow$ He Initialization

Xavier Initialisation was designed for Symmetric activation functions like sigmoid or tanh, where we need to keep forward and backward variance (Keep stable gradients). To balance both, we use n_{in} & n_{out} .

Kaiming/He Initialisation is designed for non-symmetric Activation function where it halves of the variance by "zeroing the negative values". To compensate the variance, we multiply by 2.